Foodie Hub Project Report

Overall work analysis and process functions used

1. Project Overview

Foodie Hub is a static food delivery web application prototype built with HTML, CSS, and JavaScript. The application forms a bridge between restaurant owners, customers, and delivery drivers. It demonstrates a realistic food delivery workflow where customers browse nearby restaurants, add food items to a shared cart, place orders, and track delivery status. Restaurant owners can manage menus, accept orders, assign drivers, and view analytics. Delivery drivers can update pickup and drop-off status.

The project does not use a backend server or database. Instead, it stores application state in browser localStorage. This makes the application easy to run directly from index.html while still allowing page-to-page continuity for cart, orders, restaurants, drivers, and sessions.

2. Technology Stack

- HTML: Defines separate pages for customer, restaurant, cart, orders, and driver workflows.
- CSS: Provides responsive layout, cards, forms, dashboards, order status UI, and mock map styling.
- JavaScript: Handles state management, rendering, cart behavior, checkout, order lifecycle, restaurant dashboard actions, driver actions, and simulated login/signup.
- localStorage: Stores demo data and user interactions across pages.

3. Main Files and Responsibilities

- index.html: Customer home page with nearby restaurant browsing, cuisine filters, search, recommendations, and restaurant cards.
- login.html and signup.html: Separate customer login and signup pages.
- restaurant-login.html and restaurant-signup.html: Separate restaurant owner login and onboarding pages.
- restaurant.html: Restaurant menu page with dish photos, prices, availability, reviews, and add-to-cart actions.
- cart.html: One unique shared cart and payment page for the full customer application.
- orders.html: Customer order tracking page with status steps, driver information, mock location, and dynamic ETA.
- restaurant-dashboard.html: Owner dashboard for incoming orders, menu management, order acceptance, driver assignment, and analytics.
- driver.html: Delivery driver page for pickup and drop-off status updates.

- styles.css: Complete styling for the application UI.
- app.js: Core application logic and process functions.
- README.md: Usage notes and demo account details.

4. Application Workflow

- Customer signs up or logs in through the customer authentication pages.
- Customer browses nearby restaurants on the home page and filters by cuisine.
- Customer opens a restaurant page and adds food items to the cart.
- Customer completes checkout from the shared cart and payment page.
- An order is created with status Placed and saved in localStorage.
- Restaurant owner views the order in the dashboard, accepts it, sets prep time, and assigns a driver.
- Driver views assigned orders and marks them picked up or delivered.
- Customer can track order status, driver location progress, and dynamic ETA on the orders page.

5. Data Model

The seedData object in app.js initializes the project with demo sessions, restaurants, menus, drivers, an empty cart, and an empty orders array. Restaurants include cuisine, area, distance, ETA, rating, loyalty offer, image, reviews, and menu items. Menu items include id, name, price, availability, photo, and tags. Orders include customer details, address, payment type, packaging preference, status, prep time, assigned driver, ETA, total price, and timestamps.

6. Process Functions Used in app.js

- loadState(): Loads saved data from localStorage. If no saved data exists, it initializes the application with seedData.
- saveState(state): Saves the current application state to localStorage and updates the cart count.
- getState(): Returns the latest application state.
- currency(value): Formats numeric prices as Indian rupee text.
- updateCartCount(state): Updates the cart item count shown in navigation.
- findRestaurant(state, id): Finds a restaurant by id, with a fallback to the first restaurant.
- findDish(restaurant, dishId): Finds a dish inside a restaurant menu.
- renderRestaurants(): Renders the home page restaurant cards and handles cuisine filters and search.

- renderRestaurantPage(): Renders selected restaurant details, menu items, reviews, and add-to-cart buttons.
- addToCart(restaurantId, dishId): Adds a dish to the cart or increases quantity if already present.
- cartDetails(state): Converts cart IDs into full dish and restaurant details for display and checkout.
- renderCart(): Renders cart items, quantity controls, totals, delivery details, payment options, packaging preference, and checkout order creation.
- statusIndex(status): Converts order status into a progress step index.
- orderCard(order, mode): Builds reusable order cards for customer, restaurant, and driver pages.
- orderActions(order, mode): Displays actions based on page mode, such as accepting orders, assigning drivers, rating, pickup, and delivery.
- renderOrders(): Renders customer orders and simulates driver progress for live tracking.
- bindRatingButtons(): Handles customer rating button interaction.
- renderRestaurantDashboard(): Renders incoming orders, owner menu controls, order acceptance, driver assignment, and analytics.
- renderDriverOrders(): Renders assigned driver orders and handles pickup/drop-off updates.
- mutateOrder(orderId, updater, after): Generic helper for updating an order and refreshing the current page view.
- bindAuthForms(): Simulates customer and restaurant login/signup and redirects users to the correct page.

7. Implemented Features

- Separate customer login and signup pages.
- Separate restaurant login and signup pages.
- Nearby restaurant browsing with cuisine filters.
- Restaurant menu listing with prices, dish photos, availability, ratings, and reviews.
- One shared cart and payment page.
- Online payment and cash-on-delivery options.
- Custom packaging preference.
- Order creation with customer details.
- Restaurant order acceptance and driver assignment.
- Driver pickup and drop-off workflow.
- Customer live order tracking with mock driver location and dynamic ETA.

- Basic analytics for restaurant owners.
- Loyalty and personalized recommendation-style labels.

8. Limitations

- Authentication is simulated and does not validate passwords against a backend.
- Data is stored only in browser localStorage, so it is not shared across devices.
- Mapping and driver tracking are simulated with CSS, not a real mapping API.
- Payment processing is simulated and not connected to a payment gateway.
- Push notifications are represented through UI status updates, not real notification services.
- The prompt includes automotive parts features, but the implemented application focuses on food delivery. Similar concepts such as recommendations, packaging, loyalty, and repeat ordering are represented in food-delivery form.

9. Future Enhancements

- Add backend APIs using Node.js, Express, Django, or another server framework.
- Use a database such as MongoDB, PostgreSQL, or MySQL for persistent multi-user data.
- Add real authentication and role-based authorization for customers, restaurants, and drivers.
- Integrate Google Maps or Mapbox for live driver tracking and route optimization.
- Integrate Razorpay, Stripe, or another payment gateway.
- Add real push notifications through Firebase Cloud Messaging or Web Push.
- Add restaurant image upload, menu editing forms, coupons, refunds, and admin moderation.

10. Conclusion

Foodie Hub successfully demonstrates the core workflow of a food delivery platform. It contains separate interfaces for customers, restaurant owners, and delivery drivers, while connecting them through shared localStorage state. The app includes restaurant discovery, menu browsing, cart and payment, order management, delivery assignment, driver status updates, and customer tracking. It is suitable as a frontend prototype and can be extended into a full-stack production system.